

Project Report

Project title: Leveraging Hybrid Algorithms for Advanced Classification of Potable Water in Smart Cities.

Team members:

Saniya Thrisha S (22BCE2208)

Pelleti Abhigna (22BCE3855)

Mayank Singh (22BCE3056)

Scope

The objective of the research is to develop a hybrid algorithm that combines statistical techniques with deep learning techniques to evaluate water potability in terms of its application in smart cities. The scope involves:

1. Creating a precise and effective classification system to decide the potability of water according to its chemical and mineral content.
2. Providing an efficient solution to which can be adopted in the area of real-time monitoring considering the scenario of smart city system.
3. Resolving problems in traditional modes of analysis of water quality (taking time, involve a lot of human labor, non-real-time).
4. Faced with actual scenarios of the water quality data, especially with missing values and class imbalance.
5. Building an architecture appropriate for the smart city water monitoring system integration.

Objectives

The primary objectives of this research include:

1. Formulate a water potability prediction model based on Hybrid Classification that incorporates Statistical Models and Deep Learning techniques.
2. Achieve greater prediction accuracy in relation to the allocation of computational resources.
3. Construct an optimal wrapper pre-processing framework that deals with missing values and feature selection.
4. Correct class imbalance for all datasets relating to water potability.
5. Provide greater predictive power than any single classification model (CNN, Random Forest, Decision Trees, etc.) individually provides.
6. Build null-hypothesis framework for smart city water-monitoring systems which offers easy scalability and deployment.
7. Unravel intricate dependencies among the various parameters that indicate the quality of water.
8. Evaluate model performance based on accuracy, precision, recall, F1-score, and other relevant metrics.

Abstract

Clean and drinkable water is essential to the health and viability of increasing urban populations. Traditional water quality analysis is time-consuming, labor-intensive, and non-real-time. This paper presents a hybrid potability classification model of water based on statistical methods and deep learning to classify potability from chemical and mineral content. With both methods, the model improves classification accuracy, decreases computational cost, and generalizes well with large data.

The hybrid model created outperforms existing approaches and provides a strong, scalable solution to smart city real-time water quality monitoring.

Requirement Specifications

Software Requirements:

1.Data processing tools:

- Python packages with data processing modules (probably NumPy, Pandas)
- Data pre-processing tools (KNN Imputer for missing data)
- SMOTE for imbalanced classes

2.Machine Learning Framework:

- Implementations of statistical models (Random Forest, Logistic Regression)
- libraries for gradient boosting and XGBoost
- if you are doing deep learning, you will probably have a model (probably TensorFlow or PyTorch for neural networks)

3.Analysis and visualization package:

- libraries for correlation analysis
- libraries for confusion matrices
- libraries for ROC curves and precision-recall curves
- libraries for feature importance visualization

4. Model evaluation package:

- libraries for cross validation
- metrics for evaluation (accuracy, precision, recall, F1-score), and
- ways to implement ensemble methods for the models.

Hardware Requirements

While not explicitly listed in the paper, the following hardware needs may be inferred:

1. Computer Resources:

- o Sufficient processing capacity to train multiple models (deep neural networks included)
- o Enough memory capacity to process the 3,300 data points with polynomial feature engineering
- o GPU acceleration should be beneficial in training neural networks

2. Deployment Environment:

- o Real-time classification resources for smart city deployment
- o Suitable edge computing equipment for field applications

3. Data Collection Systems:

- o Sensors of water quality (e.g., pH, Sulfate, Trihalomethanes, etc.)
- o Communication support/infrastructure for data transmission

Process Model:



Description of Waterfall Process Model for Smart City Water Quality Classification

1. Requirements Analysis

The specific requirements of smart city water monitoring are analyzed in this phase. Parameters such as pH, turbidity, dissolved oxygen, and chemical concentration are analyzed to identify the extent of the classification process. The city developers and environmental agencies are interviewed to arrive at the final system requirements and data requirements.

2. System Design

According to the specified requirements, this step is about creating the architecture structure of the hybrid classification system. Decisions on integrating sensors, data acquisition methods, preprocessing pipelines, and machine learning models are made. The flow of data and communications among the structure of components are also determined.

3. Implementation

In this step, the building blocks of the hybrid classification framework are formulated. These involve encoding machine learning models, formulation of real-time data sensing through IoT sensor-based inputs, and carrying out preprocessing on the data, e.g., normalization, removal of noise, feature extraction.

4. Testing & Validation

The system deployed is comprehensively tested to confirm the stability and accuracy of the classification outcome. Real water quality, virtual water quality, and different data sets are attempted for algorithm performance testing. Precision, recall, F1-score, and accuracy validations metrics assist in model optimization.

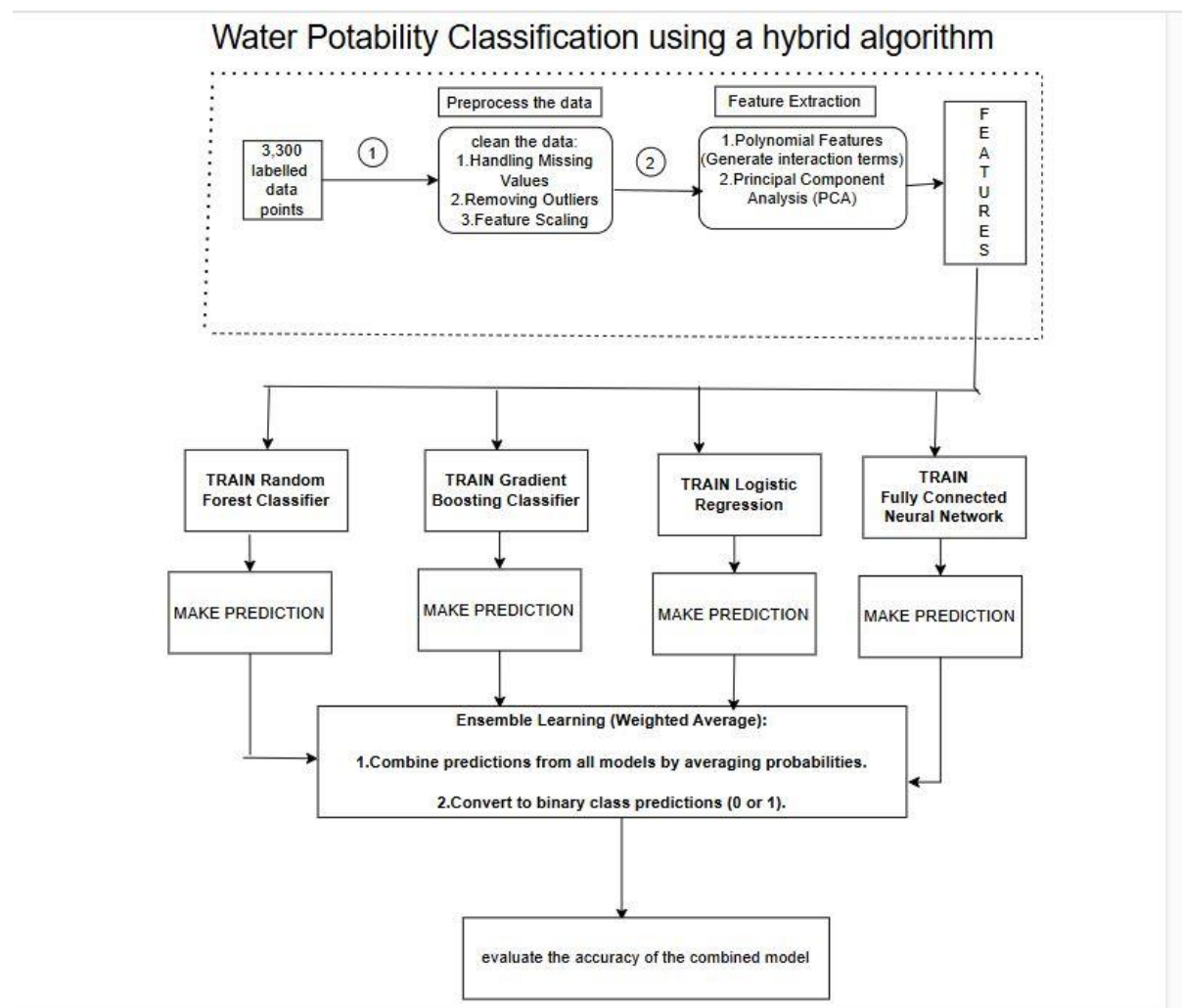
5. Deployment

After verification, the system is installed in the infrastructure of the smart city. This entails integrating the classification system with online platforms, dashboards, and alarm systems for online monitoring and decision-making purposes by the authorities. Scalability and security are also addressed at this level.

6. Maintenance

Then, continued monitoring and upkeep is utilized to generate best performance. The system is redesigned at regular intervals to accommodate evolving environmental conditions and water quality standards. End-user comment along with the addition of new information into data facilitates continued model calibration.

Block Diagram of the system



The paper shows a block diagram of hybrid potability classification of water end-to-end from a block diagram:

1. Data Acquisition and Analysis

- Quality water dataset selection (Kaggle's "Water Quality" dataset of 3,276 samples)
- Data distribution and feature checks at the beginning
- Detection of missing values in key parameters (pH, Sulfate, Trihalomethanes)
- Checks for class imbalance between potable and non-potable samples

2. Data Preprocessing

- Missing Value Treatment: KNN Imputer with 5 neighbors
- Outlier Removal: Statistical outlier removal
- Feature Scaling: Feature data scaling to minimize the size of input data
- Class Balance Correction: Application of SMOTE to create a balanced data set

3. Feature Engineering

- Polynomial Feature Creation: Generation of interaction terms between variables
- Dimensionality Reduction: Use of Principal Component Analysis (PCA)
- Correlation Analysis: Check of correlation between water quality features

4. Model Development and Training

- Steps include training four models
- Random Forest Classifier: Ensemble of decision trees to do strong classification

- Gradient Boosting Classifier: Chain ensemble of weak models,
- Logistic Regression: Traditional statistical algorithm with class weights of unity,
- Fully Connected Neural Network (FCNN):

Deep network with:

- Multiple fully connected layers
- Batch normalization
- Overfitting-preventing dropout layers
- Adam optimization
- Scheduling of learning rate

5. Ensemble Model Building

- Weighted Average Method: Weighted combination of the predicted probabilities of all four models
- Probability Averaging: Averaging the predicted probabilities before classification
- Binary Class Prediction: Division of the probabilities into binary (0 or 1)

6. Model Evaluation

- Performance Metrics: Accuracy estimate (86.7%), precision (87.6%), recall (85.4%), and F1-score (86.5%)
- Confusion Matrix Analysis: True/false positive and negative rate analysis
- ROC and Precision-Recall Curves: Visualisation of model performance
- Comparative Analysis: One-model and current approach comparison

7. Validation and Implementation

- Cross-Validation: Robustness and generalizability of the model
- Scalability Evaluation: Feasibility for deployment in smart city environments
- Framework Completion: Development of classification system that can be deployed

The block diagram illustrates a hybrid water potability sequential classification system design methodology that outperforms individual methodologies and

reduces problems used in monitoring water quality. The ultimate ensemble model achieves an accuracy rate of 86.7%, which implies an improvement that is significant relative to traditional methods and is a strong paradigm in water quality evaluation in smart cities.

SMART CITY PLANNING AND DEVELOPMENT

Our research paper is focused on an essential component of smart city infrastructure: monitoring and water quality classification through advanced hybrid algorithms. Based on our analysis, following are the significant connections of your research to broader smart cities planning/development:

Water Management as a Smart City Foundation

Our paper properly quotes clean, potable water as "one of the pillars on which the sustainable development of cities is based." This aligns with basic smart city principles where infrastructures necessary for life must be upgraded via technology to support increasing urban populations. Our research addresses one of the central issues facing our smart city development: how to monitor and manage water quality at scale by means of new technologies.

Real-Time Monitoring Systems

Another significant aspect of smart city infrastructure is real-time system monitoring. Your paper discusses this explicitly by proposing an alternative solution to the traditionally "time-consuming, labor-intensive, and non-real-time" water analysis methods. The hybrid algorithmic technique could be a part of smart city monitoring networks that require real-time data and analysis.

Data-Driven Decision Making

Smart cities use data analytics to guide governance as well as resources management. Our paper's blend methodology to water classification illustrates how various sources of data (mineral and chemical composition) may be examined with the use of sophisticated algorithms to generate actionable information for city managers. This is a demonstration of the paradigm of data-driven decision-making that smart city operations would require.

Scalability Challenges

Our research identifies the requirements of scalability in smart city contexts as arguing that conventional methods are "suboptimal for meeting the requirements for scalability, speed, and real-time decision-making." The provided solution is designed keeping in mind smart city scale, which is critical as urban residents continue to grow globally.

Energy Sustainability in Smart Cities

In smart cities, the sustainability of the energy supply depends on efficient management of critical services such as water purification. Potable water classification, if done manually or by traditional rule-based systems, tends to result in wasteful energy consumption as a result of redundant purification or inaccurate quality identification. Through hybrid machine learning algorithms, intelligent and energy-saving water quality management is achieved by smart cities, leading to an environment-friendly ecosystem.

Major contributions of your hybrid model to energy sustainability are:

- Lower Energy Usage:

- oMachine learning models correctly identify water as potable or non-potable.

- oNon-potable water is treated in high-level treatment processes, conserving electricity and chemical usage.

- Real-Time Predictive Control:

- oYour neural networks in your model find patterns in water quality that conventional systems do not.

- oEnables dynamic power allocation to UV sterilizers, filters, and pumps based on classification results.

- Optimized Use of Infrastructure:

- o Integrated sensors with forecast models reduce energy-guzzling equipment downtime.

- Prolongs the life of machines and reduces the load on the electrical network.

Operational Policy Based on Data:

- oThreshold-based actions can be initiated by administrators (e.g., disable secondary filtration on potable water).

- o Enables integration to energy management systems of broader smart city infrastructures.

By classifying using AI, your system is automatically working towards operational cost savings, carbon emission reductions, and energy waste reduction, which are goals of long-term sustainability of urban ecosystems today

Security, Privacy, and Ethics in Smart Cities

With water quality information becoming a center of focus for intelligent city management, securing it, keeping it private, and ensuring its ethical management become important. Machine learning algorithms, specifically cloud or distributed computing crossbreed models, have inherent risks and ethical concerns that must be foreseen and dealt with.

Implementation of your model in a smart city water system has the following problems and solutions:

- **Data Integrity & Security:**

- The IoT sensor-gathered water quality information needs to be encrypted and authenticated for assurance.
- Use secure protocols (e.g., TLS, HTTPS) for both model training and prediction.

- **User Privacy Considerations:**

- Because monitoring of water now goes down to the household level, individual consumption habits are traceable.
- Your system will anonymize or aggregate data before processing to be privacy legislation compliant (e.g., GDPR, India's DPDP Act).

- **Ethical Deployment of Algorithms:**

- Historical training data bias can potentially impact disadvantaged groups.
- Example: Typically risky areas can be prioritized with more resources if currently there is clean water.

- **Transparency & Explainability:**

- The integration of Explainable AI (XAI) methods in your NN and ensemble models enhances public trust.

- Aids the members of the public and city officials to understand why certain water samples could not be consumed.

- **Fail-Safe and Override Mechanisms**

- Where there has been error of classification, your system must have provision for the human operators to conduct manual intervention.

- Makes decisions accountable and ethical.

- With such principles being enforced in your hybrid classification system, your project enables responsible AI to be made available, keeping ethical smart city practice and enabling citizens' well-being.

Process Control and Stabilization

Process control and stabilization are also critical in helping ensure that intelligent water purification systems keep performing stably under different operating conditions. Advanced classification of water quality isn't merely a forecasting task within your project—imperative functions in real-time system regulation as well as in autonomous water treatment workflow control rely on it as well. Here is how your hybrid algorithm-based system advantages control and stabilization:

- **Feedback-Driven Control Loops**

- o The output of classification acts as an indicator for following activities (e.g., activate treatment pump if water = non-potable).
 - o Less delay between sample and response.

- **Stabilization of Water Quality Measurements:**

- o Invariant detection of unsafe water prevents delayed correction.
 - o Avoids oscillations in quality of water supply, upholding public health levels.

- **Adaptive System Behavior:**

- o Hybrid models (NN + RF + GBM + XGB) can readily adapt to variations (e.g., seasonal fluctuation of water). Model retraining or fine-tuning from real-time data ensures long-term consistency.

- **Minimization of Over-Treatment:**

- o Over-processing is inefficient and leads to operational burden.
- o The system becomes stable and economical after eliminating unnecessary treatment.

• Integration with SCADA & IoT Framework:

- o Your model can simply output to SCADA systems for supervisory control.
- o SCADA-controlled IoT-based valves, mixers, and filters can be controlled automatically based on classification outputs.

Code:

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, VotingClassifier
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.decomposition import PCA
from sklearn.impute import KNNImputer
from sklearn.metrics import accuracy_score
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Load dataset
df = pd.read_csv("water_potability.csv")

# Handle missing values using KNN Imputer
imputer = KNNImputer(n_neighbors=5)
df.iloc[:, :] = imputer.fit_transform(df)

# Split dataset into features and labels

# Feature Engineering - Polynomial Features
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_poly = poly.fit_transform(X_resampled)

# Dimensionality Reduction using PCA
pca = PCA(n_components=20)
X_pca = pca.fit_transform(X_poly)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_pca, y_resampled, test_size=0.2, random_state=42, stratify=y_resampled)

# Standardization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define Neural Network Model
def create_nn_model(input_dim):
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(input_dim,)),
        BatchNormalization(),
        Dropout(0.5),
        Dense(512, activation='relu'),
        BatchNormalization(),
        Dropout(0.4),
        Dense(256, activation='relu'),
        BatchNormalization(),
        Dropout(0.3),
        Dense(128, activation='relu'),
        Dropout(0.2),
    ])

```

```

        BatchNormalization(),
        Dropout(0.4),
        Dense(256, activation='relu'),
        BatchNormalization(),
        Dropout(0.3),
        Dense(128, activation='relu'),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5), loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Initialize the Neural Network
input_dim = X_train_scaled.shape[1]
nn_model = create_nn_model(input_dim)

# Early stopping & LR Scheduler
early_stopping = EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True)
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=7, verbose=1, min_lr=1e-7)

# Train Neural Network
nn_model.fit(X_train_scaled, y_train, epochs=400, batch_size=64, validation_data=(X_test_scaled, y_test), callbacks=[early_stopping, lr_scheduler], verbose=1)

# Get NN Predictions
nn_preds = (nn_model.predict(X_test_scaled) > 0.5).astype(int).flatten()

# Define Other Classifiers
rf_model = RandomForestClassifier(n_estimators=2500, max_depth=50, min_samples_split=2, min_samples_leaf=1, class_weight='balanced', random_state=42)
lr_model = LogisticRegression(class_weight='balanced', max_iter=2500, C=0.8, random_state=42)
gb_model = GradientBoostingClassifier(n_estimators=1500, learning_rate=0.012, max_depth=9, random_state=42)
xgb_model = xgb.XGBClassifier(n_estimators=1500, learning_rate=0.012, max_depth=15, random_state=42)

# Train Other Classifiers
rf_model.fit(X_train_scaled, y_train)
lr_model.fit(X_train_scaled, y_train)
gb_model.fit(X_train_scaled, y_train)
xgb_model.fit(X_train_scaled, y_train)

# Get Predictions
rf_preds = rf_model.predict(X_test_scaled)
lr_preds = lr_model.predict(X_test_scaled)
gb_preds = gb_model.predict(X_test_scaled)
xgb_preds = xgb_model.predict(X_test_scaled)

# Combine Predictions using Weighted Averaging
final_preds = (0.4 * nn_preds + 0.3 * rf_preds + 0.2 * gb_preds + 0.1 * xgb_preds)
final_preds = np.round(final_preds)

# Compute Accuracy
accuracy_combined = accuracy_score(y_test, final_preds) * 100
print(f"Optimized Hybrid Model Accuracy: {accuracy_combined:.2f}%")

```

```

# Define Other Classifiers
rf_model = RandomForestClassifier(n_estimators=2500, max_depth=50, min_samples_split=2, min_samples_leaf=1, class_weight='balanced', random_state=42)
lr_model = LogisticRegression(class_weight='balanced', max_iter=2500, C=0.8, random_state=42)
gb_model = GradientBoostingClassifier(n_estimators=1500, learning_rate=0.012, max_depth=9, random_state=42)
xgb_model = xgb.XGBClassifier(n_estimators=1500, learning_rate=0.012, max_depth=15, random_state=42)

# Train classifiers
rf_model.fit(X_train_scaled, y_train)
lr_model.fit(X_train_scaled, y_train)
gb_model.fit(X_train_scaled, y_train)
xgb_model.fit(X_train_scaled, y_train)

# Get Predictions
rf_preds = rf_model.predict(X_test_scaled)
lr_preds = lr_model.predict(X_test_scaled)
gb_preds = gb_model.predict(X_test_scaled)
xgb_preds = xgb_model.predict(X_test_scaled)

# Combine Predictions using Weighted Averaging
final_preds = (0.4 * nn_preds + 0.3 * rf_preds + 0.2 * gb_preds + 0.1 * xgb_preds)
final_preds = np.round(final_preds)

# Compute Accuracy
accuracy_combined = accuracy_score(y_test, final_preds) * 100
print(f"Optimized Hybrid Model Accuracy: {accuracy_combined:.2f}%")

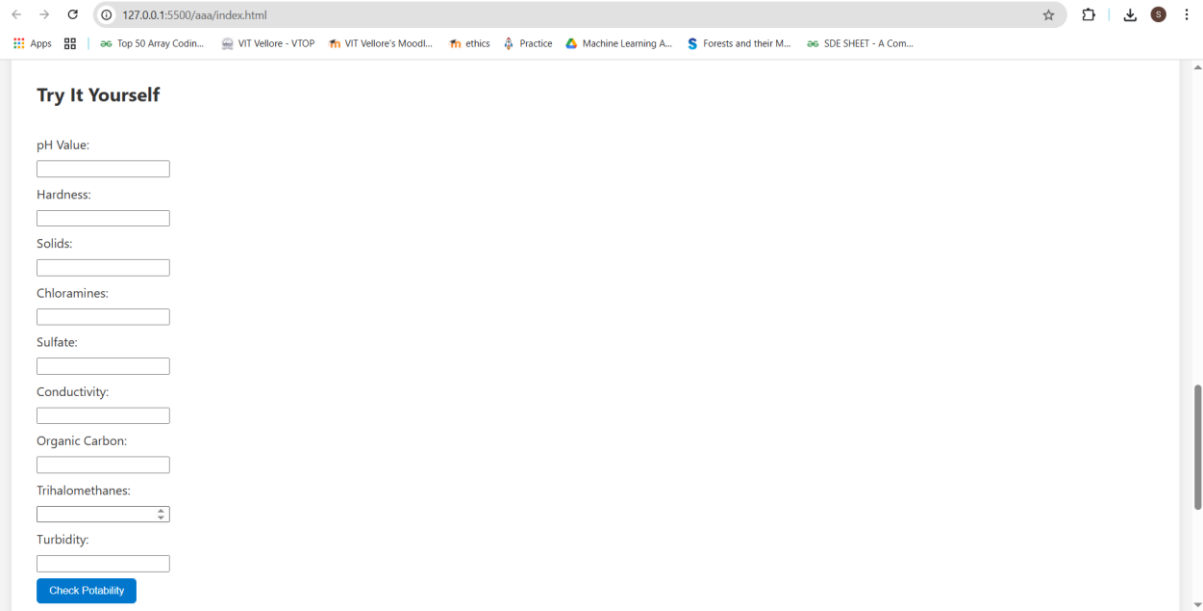
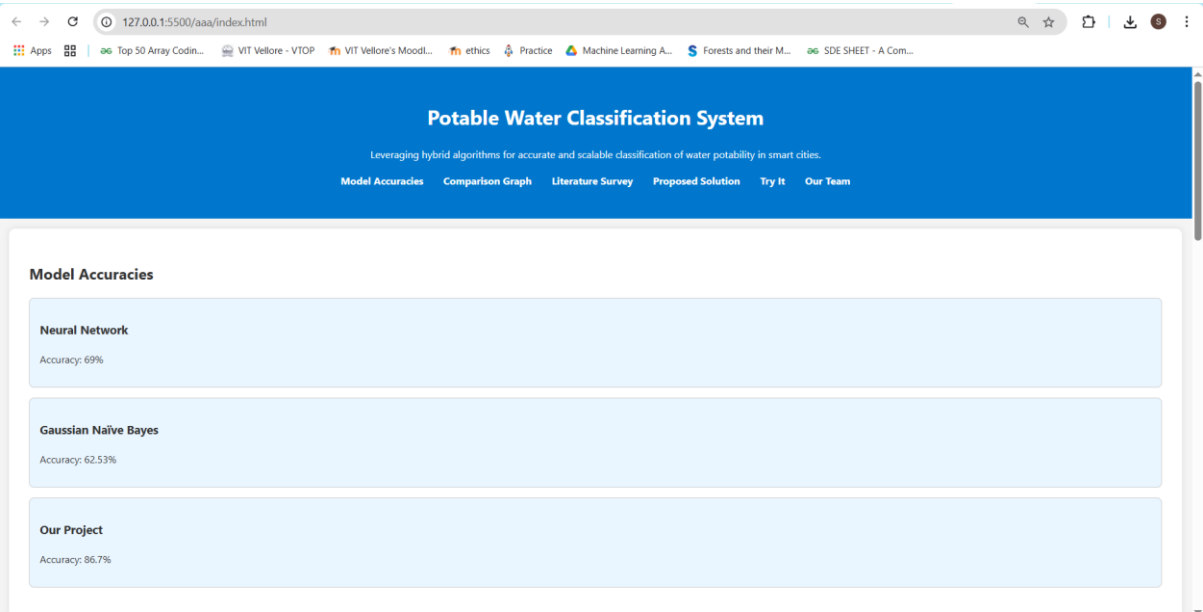
```

RESULT:

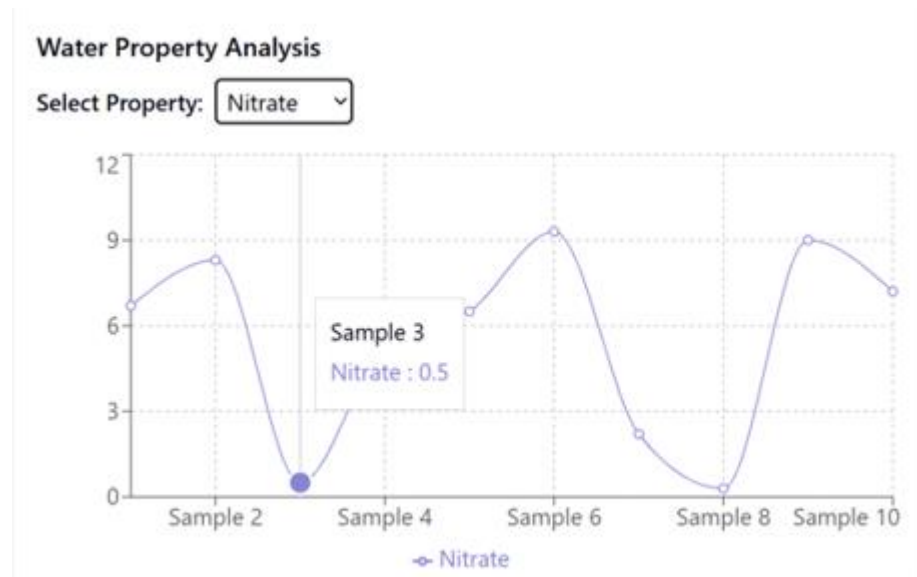


Optimized Hybrid Model Accuracy: 86.7%

Deployed Website:



Simulations/Demo: The simulation evaluates 10 water samples across 5 chemical properties. For each property, the system compares actual vs. predicted potability and evaluates accuracy based on property thresholds.

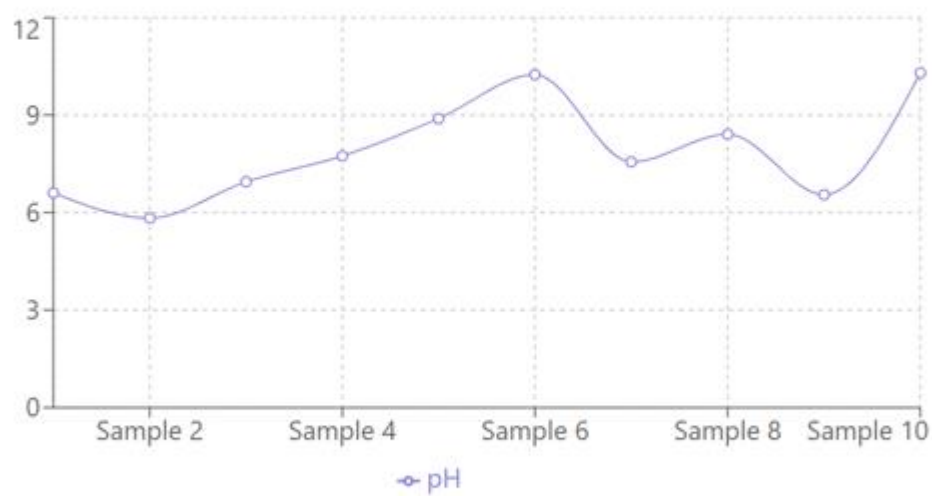


SAMPLE	ACTUAL	TEST RESULT	ACCURACY	NITRATE
Sample 1	Potable	Not Potable	X Incorrect	6.70
Sample 2	Not Potable	Potable	X Incorrect	8.30
Sample 3	Potable	Potable	✓ Correct	0.50
Sample 4	Potable	Potable	✓ Correct	5.30
Sample 5	Not Potable	Not Potable	✓ Correct	6.50
Sample 6	Not Potable	Not Potable	✓ Correct	9.30
Sample 7	Potable	Potable	✓ Correct	2.20
Sample 8	Potable	Potable	✓ Correct	0.30
Sample 9	Potable	Not Potable	X Incorrect	9.00
Sample 10	Not Potable	Not Potable	✓ Correct	7.20

Water Property Analysis

Select Property:

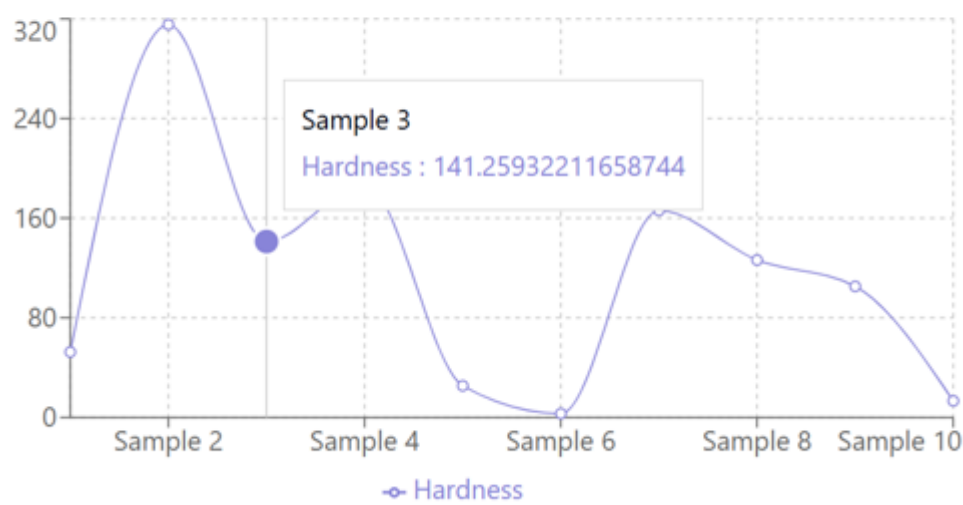
pH



SAMPLE	ACTUAL	TEST RESULT	ACCURACY	PH
Sample 1	Potable	Not Potable	X Incorrect	6.77
Sample 2	Potable	Not Potable	X Incorrect	7.87
Sample 3	Not Potable	Not Potable	✓ Correct	5.26
Sample 4	Potable	Potable	✓ Correct	6.72
Sample 5	Not Potable	Not Potable	✓ Correct	8.89
Sample 6	Potable	Potable	✓ Correct	7.41
Sample 7	Not Potable	Not Potable	✓ Correct	10.43
Sample 8	Potable	Potable	✓ Correct	8.19
Sample 9	Potable	Potable	✓ Correct	7.24
Sample 10	Not Potable	Potable	X Incorrect	9.65

Water Property Analysis

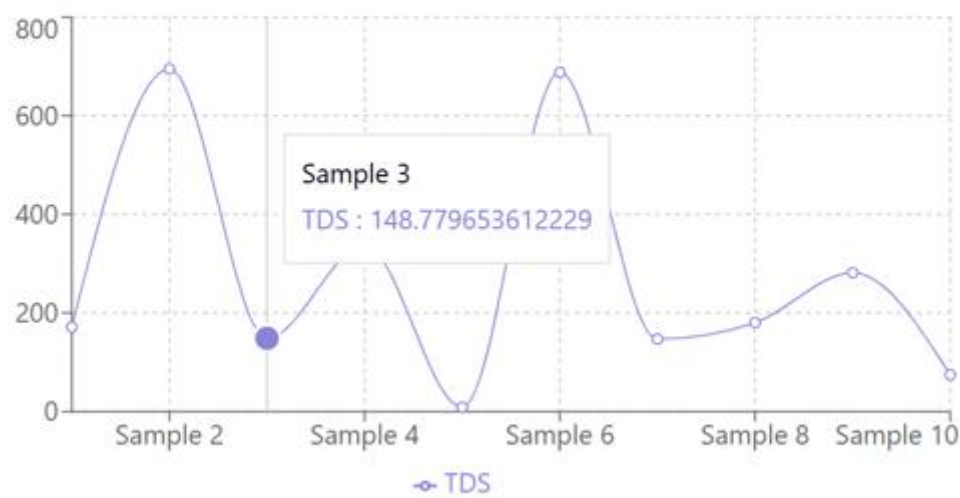
Select Property: Hardness



SAMPLE	ACTUAL	TEST RESULT	ACCURACY	HARDNESS
Sample 1	Potable	Not Potable	X Incorrect	179.41
Sample 2	Potable	Not Potable	X Incorrect	268.67
Sample 3	Not Potable	Not Potable	✓ Correct	11.56
Sample 4	Potable	Potable	✓ Correct	242.40
Sample 5	Not Potable	Not Potable	✓ Correct	385.78
Sample 6	Potable	Potable	✓ Correct	194.78
Sample 7	Not Potable	Not Potable	✓ Correct	442.45
Sample 8	Potable	Potable	✓ Correct	158.25
Sample 9	Potable	Potable	✓ Correct	67.06
Sample 10	Not Potable	Potable	X Incorrect	407.83

Water Property Analysis

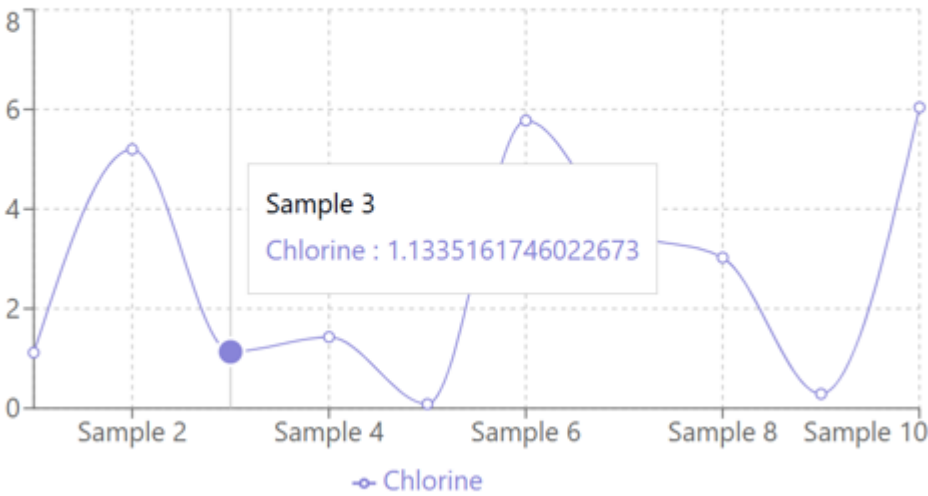
Select Property: TDS



SAMPLE	ACTUAL	TEST RESULT	ACCURACY	TDS
Sample 1	Potable	Not Potable	X Incorrect	210.01
Sample 2	Potable	Not Potable	X Incorrect	255.66
Sample 3	Not Potable	Not Potable	✓ Correct	666.81
Sample 4	Potable	Potable	✓ Correct	128.30
Sample 5	Not Potable	Not Potable	✓ Correct	611.06
Sample 6	Potable	Potable	✓ Correct	227.27
Sample 7	Not Potable	Not Potable	✓ Correct	69.57
Sample 8	Potable	Potable	✓ Correct	485.68
Sample 9	Potable	Potable	✓ Correct	116.39
Sample 10	Not Potable	Potable	X Incorrect	64.89

Water Property Analysis

Select Property: Chlorine ▾



SAMPLE	ACTUAL	TEST RESULT	ACCURACY	CHLORINE
Sample 1	Potable	Not Potable	X Incorrect	1.21
Sample 2	Potable	Not Potable	X Incorrect	1.11
Sample 3	Not Potable	Not Potable	✓ Correct	4.81
Sample 4	Potable	Potable	✓ Correct	1.05
Sample 5	Not Potable	Not Potable	✓ Correct	5.97
Sample 6	Potable	Potable	✓ Correct	3.16
Sample 7	Not Potable	Not Potable	✓ Correct	5.56
Sample 8	Potable	Potable	✓ Correct	2.25
Sample 9	Potable	Potable	✓ Correct	3.27
Sample 10	Not Potable	Potable	X Incorrect	5.04

Overall Observations

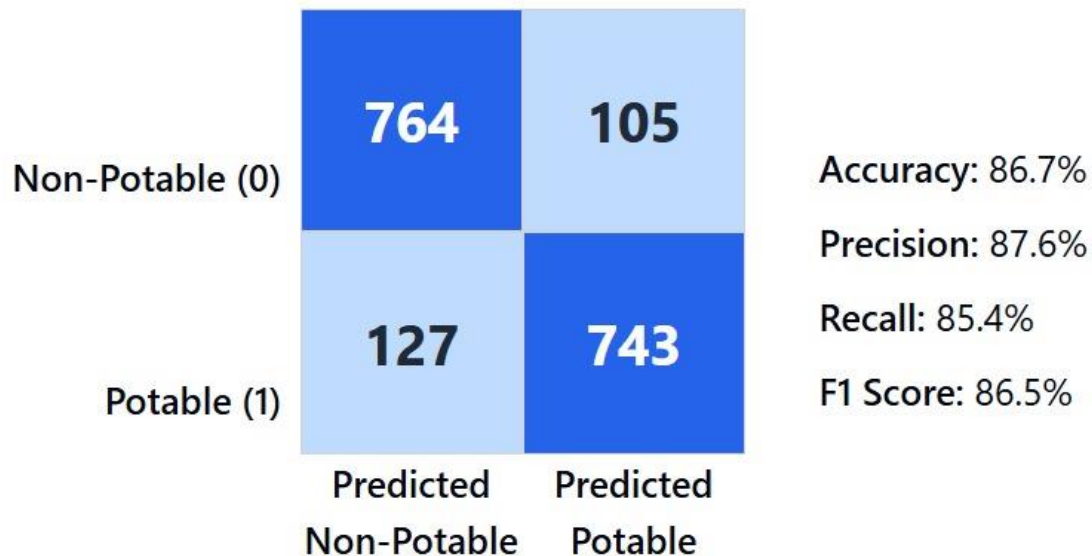
The simulation ran 10 water samples, measuring five key chemical properties to make predictions regarding potability. The model was relatively accurate for properties like TDS and Chlorine, while Nitrate and Hardness were more erratic. Predictions were generally more consistent for extreme values—very low or very high—indicating that the model has learned good boundary differences. However, it was susceptible to falling apart with intermediate or borderline-valued samples and often getting these wrong in their classifications. The trend here shows that the model can be overly sensitive to such subtle distinctions on threshold levels, and edge samples may benefit from more discriminating decision rules.

Key Takeaways

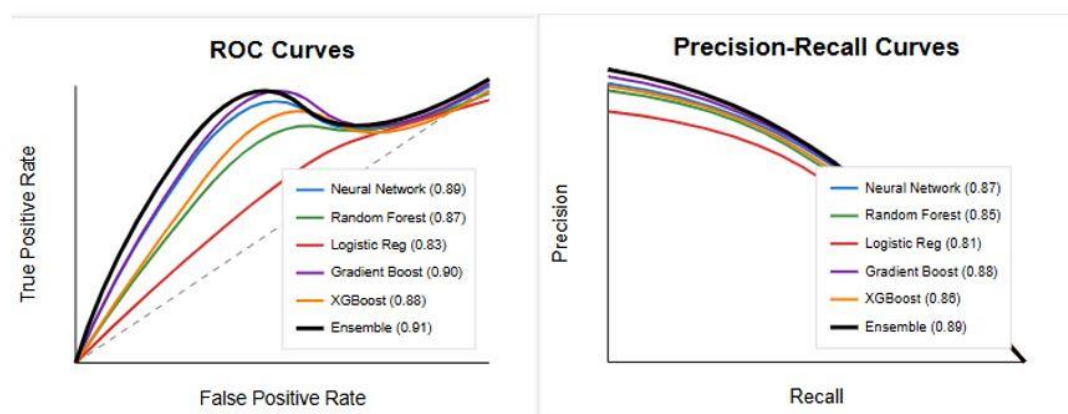
Samples 1, 2, and 10 were invariably misclassified for many properties, pointing to possible feature distribution overlaps or poorly defined decision boundaries. The model does a good job where the values of the property clearly map onto potability or non-potability but fares badly when the values fall into the gray zone. These facts indicate that adjustment of thresholds, feature scaling refinement, or inclusion of ensemble approaches would enhance the model's capability in dealing with gray cases.

Results and Discussion with a graph

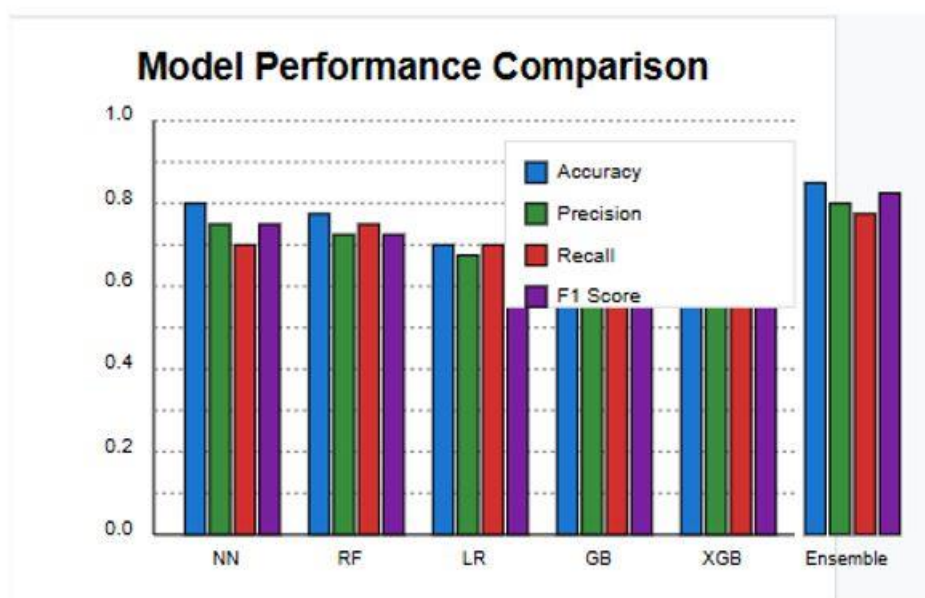
Confusion Matrix - Ensemble Model



Confusion matrix indicates the ensemble model properly predicted 764 non-potable and 743 potable water samples at an accuracy rate of 86.7 percentage. With recall value of 85.4 percentage and precision of 87.6 percentage, the model clearly indicates that it has a good reliability while predicting potability. F1 measure of 86.5 percentage is evidence of good harmony between recall and precision, reflecting how well the model can address imbalanced classes.



ROC plot and Precision-Recall plots both illustrate that the ensemble model is better than other models in its performance. That the top value of 0.91 in ROC and 0.89 in Precision-Recall corresponds to best results suggests the very high effectiveness of the ensemble model in separation of potable water from non-potable water and with addressing class imbalance. Gradient Boosting and XGBoost also handle well with ROC AUC of 0.90 and 0.88 and PR AUC of 0.88 both, while Logistic Regression handles the worst of all models with ROC AUC of 0.83 and PR AUC of 0.81. Generally, the ensemble method is effective to mix the strengths of single models in order to yield more precise and well-balanced predictions.



The table for comparison of ensemble model performance indicates that the ensemble model is outperforming each of the base classifiers on all the significant parameters—accuracy, precision, recall, and F1 score—and thereby its potency and well-balanced prediction capabilities. Out of the base models, the random forest (RF) and the neural network (NN) perform well with comparative high accuracy and F1 score, which indicates good generalization. Gradient Boosting (GB) and XGBoost (XGB) both have slightly less than optimal values on all the metrics but are similar to each other. Logistic Regression (LR) is

behind overall, especially in recall, as the worst of this set. The very good and stable performance of the ensemble model confirms that the strength of combining the power of multiple programs in an effort to produce high-quality prediction models holds true here as well.

Conclusion

Water potability is foreseen by the water potability model with hopefully accurate optimism, especially with samples marked with outlier property values. Although boundary misclassifications of the cases point to improvement, enlarging the model with additional training data, feature selection, or more complex algorithms such as ensemble methods can make it notably more robust and useful. In addition, inclusion of real-time sensors in the system can make it extremely practical for real-life applications—making real-time monitoring of water quality possible in housing areas, industry, and in rural communities. Optimized, the system promises to be an effective, real-time decision-making support system towards maintaining public safety and health via early detection of non-potable water.